

MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR

Department of Chemical Engineering



Artificial Intelligence and Machine Learning in Chemical Engineering

Project Assignment

Machine Learning Model on Vacuum Membrane Distillation

—

Submitted On: November 15, 2025

Submitted by:

Ishaan Ghosh (2023uch1872)
Isha Joshi (2023uch1874)
Muskan Sharma (2023uch1883)
Puneet Kumar Jha (2023uch1887)

ABSTRACT

Vacuum Membrane Distillation (VMD) is a thermally driven separation process which makes use of a membrane made up of hydrophobic material to create an interface between liquid and vapor. The driving force in VMD is the partial pressure difference of vapor across the membrane. Due to pressure difference, the vapor is able to cross the membrane and collected using vacuum and condensed to give pure liquid making it ideal for purification of water from salts and minerals. To demonstrate the working of VMD in desalination of water, supervised machine learning algorithms such as Artificial Neural Networks, Support Vector Regression etc. were applied by optimizing parameters such as feed temperature, flow rate, absolute pressure and degree of salination. Input and output values (permeate flux, output ratio) were used to determine hyperparameters for the best fit. Support Vector Regressor provided the best fit for testing data.

PROBLEM STATEMENT

Development of Machine learning model for Vacuum Membrane Distillation (VMD).

OBJECTIVE

To compare the working of Support Vector Regressor and Multi Perceptron Regressor for desalination using VMD.

INTRODUCTION

Artificial Intelligence and Machine Learning (AI & ML) is an emerging field. It has extensive applications in different fields of science and engineering. Its flexibility is not only limited to

computer science and data science analysis only. Machine learning can be utilized in numerous different sectors ranging from healthcare to scientific research. Model training and evaluation can be used in various aspects such as developing image recognition, processing, detailed search etc. The various tools and solutions offered by machine learning are of great significance for the progress of any industry.

Thus, it is intuitive to be able to find ways in which its potential can be harnessed to provide simpler and efficient solutions to pre-existing problems and give rise to new technologies which can be cost effective as well reduce environmental impacts at the same time optimizing the process itself without much human intervention.

Machine Learning (ML) can be broadly classified into Supervised and Unsupervised learning. There are different algorithms in both kinds of learnings. Under supervised machine learning, model is trained and evaluated using data points with a definite result such as identification of a fruit, animal or predicting the value of a configuration depending upon various trends. The different supervised learning algorithms include Linear Regression, Gradient Descent, Support Vector Machines, Neural Networks etc. They can be employed with different combinations of other algorithms as well to build a model and testing it with data set within or out of experimental run.

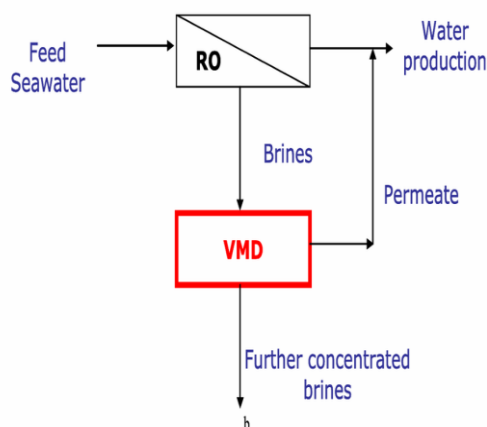
Supervised machine learning is therefore a novel approach in the Chemical Engineering field with its various numerical and analytically complex processes and operations including VMD because it can be used to simulate

different processes such as heat transfer and mass transfer to obtain different solutions, results and trends of a process. It can be used to determine the factors which are more dominant in comparison to others while operation of a process. It can be used to highlight and detect various control measures which can be a crucial part of developing a more refined operability from the use of classification and regression modules. It can be a good source of reducing the mathematical complexity underlying the processes and increase the workability without needing to physically emulate the whole experimental apparatus. As one of the fundamentals of machine learning is that a machine learning algorithm should learn from itself, it can give rise to increased performance, reduced learning rates, less underfittings and overfittings along with more accuracy.

THEORETICAL BACKGROUND

Vacuum membrane distillation is a thermal separation process that uses a hydrophobic microporous membrane and a vacuum to separate vapor from a liquid mixture. It operates at a lower operating temperature, lower hydrostatic pressure and less demanding membrane mechanical properties. Hydrophobic membrane acts as a physical barrier between hot feed

solution and vacuum side, only vapor molecules can pass through membrane pores. A vacuum is applied on permeate side to maintain a low vapor pressure which leads to enhancing the driving force for vapor transport. Vapor is then condensed externally (purified product). VMD permits higher partial pressure gradient and hence this has higher permeate flux. In this technique low thermal and concentration boundary layers form on the permeate side of membrane. operational parameters used in this technique are feed temperature, vacuum pressure and feed salinity and outputs are permeate flux, gained output ratio and specific thermal temperature.



Key parameters influencing flux:

1. **Feed temperature:** On increasing temperature VP increases and flux increases.
2. **Vacuum pressure:** On decreasing permeate pressure driving force increases and flux increases.

3. **Feed flow rate:** On increasing flow temperature polarization decreases and flux increases.
4. **Membrane pore size:** Larger pores have low mass transfer resistance and thus increase in flux.
5. **Membrane thickness:** Thinner membranes have less resistance hence increase in flux.
6. **Thermal conductivity:** Higher K leads to more heat loss and less efficiency.
7. Permeate flux is inversely proportional to thermal energy consumption. Thus, increase in flux is ideal for saving energy.

Mass and Heat Transfer Mechanism

Vapor generation occurs at the feed membrane interface. The vapor then diffuses through the membrane pores by Knudsen diffusion or molecular diffusion, depending on pore size and pressure conditions. The overall transport resistance is a combination of:

1. Evaporation resistance at the feed interface,
2. Diffusion resistance inside the membrane pores, and
3. Condensation resistance on the permeate side.

The temperature difference between the bulk feed and membrane surface due to latent heat of vaporization is termed temperature polarization, which can reduce flux.

Proper hydrodynamic design and module configuration help minimize this effect.

RESULTS AND DISCUSSIONS

For the development of a machine learning model for Vacuum membrane Distillation, parameters like Operating Temperature(T), Permeate Flux(F), Vacuum side pressure (P) and Feed flow rate (Q). This is a regression type problem under supervised machine learning. The data set for regressive analysis was obtained from the research paper (Iqbal et al., 2024) and it constitutes the following data.

T	P	Q	F
60	12	2	1.5
62.5	12	2	1.98
65	12	2	2.45
67.5	12	2	2.93
70	12	2	3.4
60	12	2.5	1.8
62.5	12	2.5	2.3
65	12	2.5	2.8
67.5	12	2.5	3.3
70	12	2.5	3.8
60	12	3	1.95
62.5	12	3	2.46
65	12	3	2.98
67.5	12	3	3.49
70	12	3	4
60	12	3.5	1.9
62.5	12	3.5	2.53
65	12	3.5	3.15
67.5	12	3.5	3.78
70	12	3.5	4.4
60	12	3.5	1.8
62.5	12	3.5	2.45
65	12	3.5	3.1
67.5	12	3.5	3.75
60	15	3.5	1.2
62.5	15	3.5	1.7
65	15	3.5	2.2
67.5	15	3.5	2.7
70	15	3.5	3.4
60	18	3.5	0.3
62.5	18	3.5	0.9
65	18	3.5	1.5

67.5	18	3.5	2.1
70	18	3.5	2.7
60	12	2.25	1.58
60	12	2.5	1.65
60	12	3	1.72
65	12	2.25	2.56
65	12	2.5	2.68
65	12	3	2.85
65	12	3.5	3
70	12	2	3.9
70	12	2.25	4
70	12	2.5	4.15
70	12	3	4.3
70	12	3.5	4.55
70	12	2	3.5
70	12	2.25	3.68
70	12	2.5	3.85
70	12	3	4.1
70	12	3.5	4.45
70	15	2	2.85
70	15	2.25	2.95
70	15	2.5	3.05
70	15	3	3.2
70	15	3.5	3.35
70	18	2	2.3
70	18	2.25	2.4
70	18	2.5	2.5
70	18	3	2.7
70	18	3.5	2.65

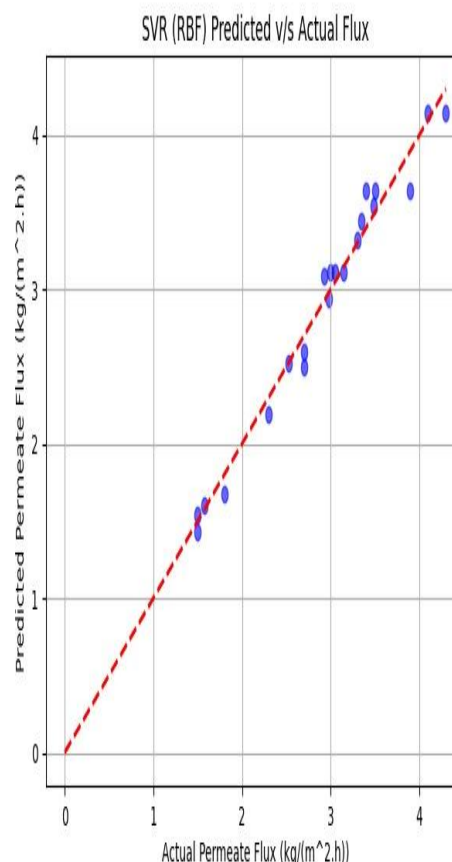
To perform regression analysis on the above-mentioned dataset, Support Vector Machines and Multilayer perceptron was used.

Support Vector Regressor is a module under Support Vector Machines used to perform regression tasks by finding the best fit with a continuous output as well by maintaining some specific level of tolerance. SVR is robust against and overfitting and effective with limited experimental data points because the solution is defined by Support Vectors. It can also be used for non-linear mapping with the help of Kernels by mapping the input features to a high dimensional feature space where correlations can be modelled linearly.

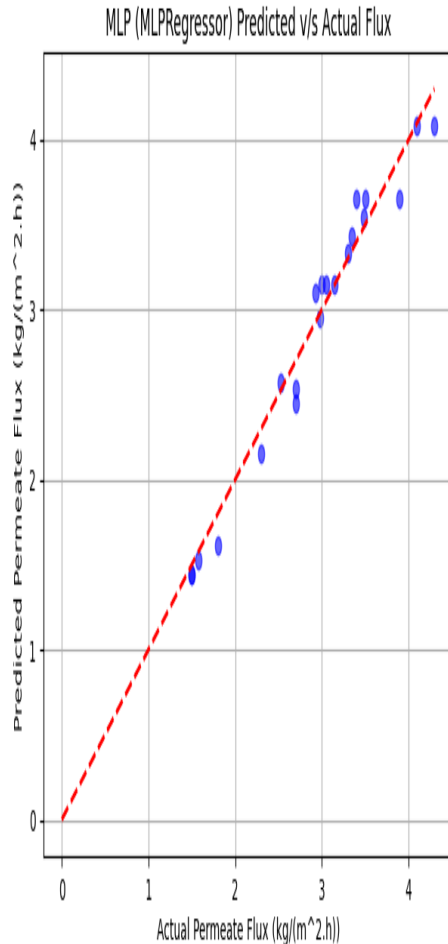
Multilayer Perceptron is an example of Artificial Neural Network (ANN) which consists of an input layer, one or more hidden layers consisting of several neurons connected with previous and next layers, along with an output layer which is used to determine the result. When data moves forward, neurons apply weights, bias and activation function before transferring it to the next layer. Continuous results are produced by using a linear activation function. Backpropagation algorithm along with gradient descent is used to optimize weights and bias and reducing the loss functions making it extremely flexible for complex and non-linear data.

While using Support Vector Regressor different modules were imported from Sklearn library such as metrics, preprocessing, StandardScaler etc. A dataframe was created with features such as T,P and Q with the help of Pandas Dataframe. The target variable F (Permeate flux) was used to determine the performance of SVR. Two-third of the available data was used for training and the rest for testing. Under SVR, Radial Basis Kernel (RBF) was trained using cross validation and the performance of the model was estimated using R2 score, RMSE (Root mean squared error) and MAE (Mean absolute error). After fitting the data, the actual value of permeated flux was plotted against

the predicted value. Different range of accuracies were obtained by changing the random state of training and testing with the maximum accuracy being 94 percent in the case of random_state = 42. The following curve belongs to this particular run.



While making use of the Multilayer Perceptron (MLP), the same initial procedure as SVR was followed. For the MLP regressor Adam solver was used with 5000 number of iterations to evaluate the best fit. The performance was measured in terms of the previously used metrics R2 score, RMSE and MAE. The plot was obtained in this case as following.



value of error as compared to Multilayer Perceptron. Though the difference between the performance of both models is very less, we can conclude the Support Vector Spotted Hyena Regressor is more suitable for analyzing Vacuum Membrane Distillation for desalination of water.

REFERENCES

(Al-Sairfi et al., 2024)
 (Ismael et al., 2023)
 (Mericq et al., 2010)
 (Baghel et al., 2020)
 (Iqbal et al., 2024)

CONCLUSIONS

For Support Vector Regressor:

1. $R^2 = 0.976647$
2. $RMSE = 0.121482$
3. $MAE = 0.099582$

For Multilayer Perceptron Regressor:

1. $R^2 = 0.970376$
2. $RMSE = 0.136824$
3. $MAE = 0.106236$

On comparing the three parameters we can observe that the performance of Support vector regressor was more optimized and had less

APPENDIX

Support Vector Regressor

```
import pandas as pd
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('master_dataset.csv')
X = df[['T','P','Q']]
Y = df['F']
scaler_X = StandardScaler()
scaler_Y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
Y_scaled = scaler_Y.fit_transform(Y.values.reshape(-1,1)).ravel()
X_train, X_test, Y_train, Y_test =
train_test_split(X_scaled,Y_scaled,test_size=0.33,random_state=42)
#Training the RBF kernel using k folds cross validation
param_grid = {
    'C':[10,100,500],
    'gamma':['scale',0.01,0.1],
    'epsilon':[0.001,0.01,0.1]
}
svr = SVR(kernel = 'rbf')
grid = GridSearchCV(svr,param_grid,cv = 5,scoring = 'r2')
grid.fit(X_train,Y_train)
best_model = grid.best_estimator_

Y_pred_scaled = best_model.predict(X_test)
Y_test_orig = scaler_Y.inverse_transform(Y_test.reshape(-1,1)).ravel()
Y_pred_orig = scaler_Y.inverse_transform(Y_pred_scaled.reshape(-1,1)).ravel()
r2 = r2_score(Y_test_orig,Y_pred_orig)
rmse = (mean_squared_error(Y_test_orig,Y_pred_orig))**(1/2)
mae = mean_absolute_error(Y_test_orig,Y_pred_orig)

print(f'R^2 = {r2:5f}, RMSE = {rmse:5f}, MAE = {mae:5f}')

plt.figure(figsize = (7,5))
plt.scatter(Y_test_orig, Y_pred_orig, c = 'blue', alpha = 0.6)
plt.plot([0,max(Y_test_orig)], [0,max(Y_test_orig)], 'r--', lw=2)
```

```
plt.xlabel('Actual Permeate Flux (kg/(m^2.h))')
plt.ylabel('Predicted Permeate Flux (kg/(m^2.h))')
plt.title('SVR (RBF) Predicted v/s Actual Flux')
plt.grid(True)
plt.show()
```

Multilayer Perceptron: ANN Regressor

```
import pandas as pd
import numpy as np
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
```

```
df = pd.read_csv('master_dataset.csv')
X = df[['T', 'P', 'Q']]
Y = df['F']
scaler_X = StandardScaler()
scaler_Y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
Y_scaled = scaler_Y.fit_transform(Y.values.reshape(-1,1)).ravel()
X_train, X_test, Y_train, Y_test =
train_test_split(X_scaled, Y_scaled, test_size=0.33, random_state=42)
```

```
param_grid = {
    'hidden_layer_sizes': [(32, 16), (50, ), (64, 32)],
    'activation': ['relu', 'tanh'],
    'alpha': [1e-4, 1e-3, 1e-2],
    'learning_rate_init': [1e-3, 1e-2],
}
mlp = MLPRegressor(solver='adam', max_iter=5000, random_state=42,
early_stopping=True)
grid = GridSearchCV(mlp, param_grid, cv=5, scoring='r2', n_jobs=-1, verbose=0)

grid.fit(X_train, Y_train)
best_model = grid.best_estimator_
```

```
Y_pred_scaled = best_model.predict(X_test)
Y_test_orig = scaler_Y.inverse_transform(Y_test.reshape(-1,1)).ravel()
Y_pred_orig = scaler_Y.inverse_transform(Y_pred_scaled.reshape(-1,1)).ravel()
```

```
r2 = r2_score(Y_test_orig,Y_pred_orig)
rmse = (mean_squared_error(Y_test_orig,Y_pred_orig))**(1/2)
mae = mean_absolute_error(Y_test_orig,Y_pred_orig)

print(f'R^2 = {r2:5f}, RMSE = {rmse:5f}, MAE = {mae:5f}')

plt.figure(figsize = (7,5))
plt.scatter(Y_test_orig, Y_pred_orig, c = 'blue', alpha = 0.6)
plt.plot([0,max(Y_test_orig)], [0,max(Y_test_orig)], 'r--', lw=2)
plt.xlabel('Actual Permeate Flux (kg/(m^2.h))')
plt.ylabel('Predicted Permeate Flux (kg/(m^2.h))')
plt.title('MLP (MLPRegressor) Predicted v/s Actual Flux')
plt.grid(True)
plt.show()
```